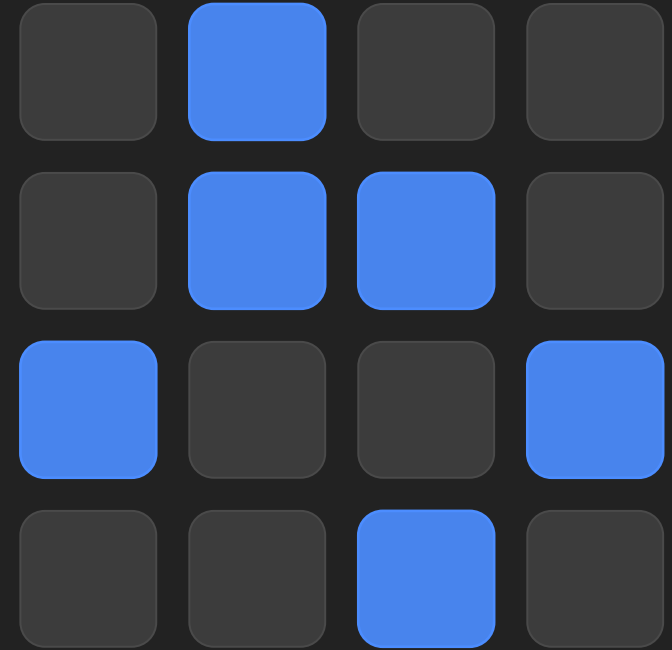




Robot Visions

Breaking reCAPTCHA at Zero Cost



No paid APIs. No solver farms. No GPU. **Just open source AI vision models on a local machine.**

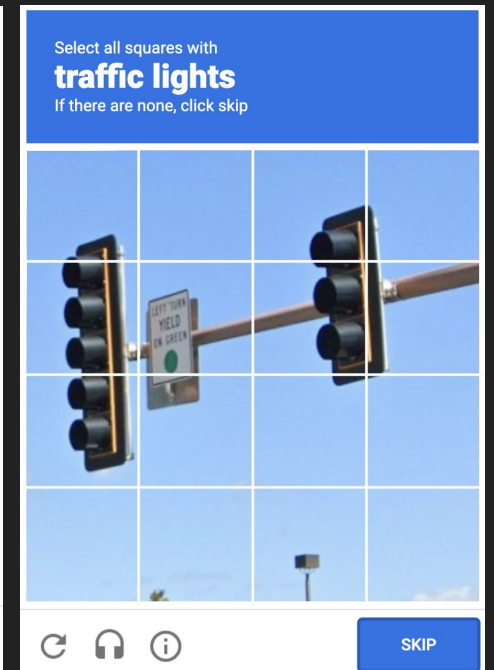
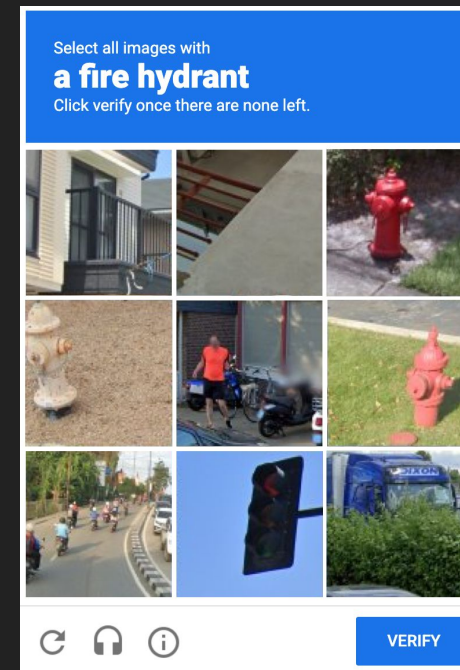
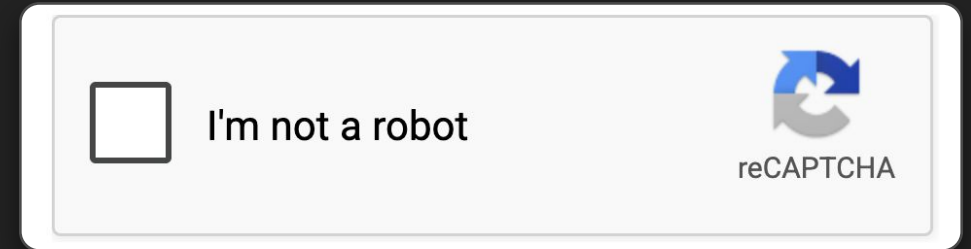
Suphannee Sivakorn

WE'VE ALL BEEN HERE

You know this box. You hate this box.

reCAPTCHA is the bouncer at the door of basically the entire web. For years, it has mostly worked.

Then we gave everyone AI.



I helped break this once already.

2016 · EuroS&P / Black Hat Asia

- First deep learning-based attack on Google reCAPTCHA
- Classified via a paid, CNN-based service API
- 70.78% accuracy · ~19s per challenge
- Also mapped the risk engine & fingerprinting

2026 · SummerCon (This Talk)

- Open source, local, zero-shot models — no training
- \$0 cost · no API · no GPU required
- And a zero-CODE path: a plain-English file
- Personal project — done independently

Ten years of “improvements” later — the attacker’s job got cheaper.

WHY THIS MATTERS

reCAPTCHA guards almost the entire web.

5.6M+

Companies using reCAPTCHA

99.9%

Market Share

#1

reCAPTCHA is the dominant service

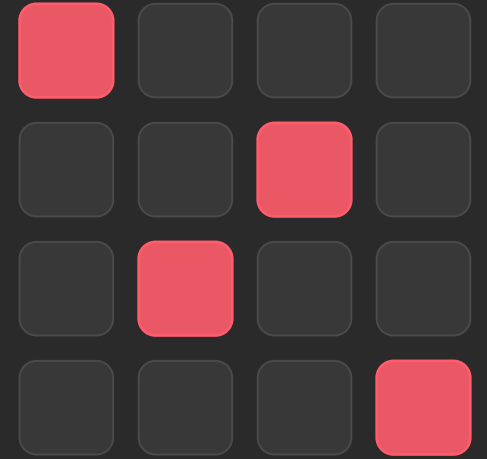
What it's supposed to stop:

- Credential stuffing
- Bulk fake-account creation
- Brute-force prevention
- Content & data scraping
- Automated form / comment spam
- Fraud mitigation

Statistics source: 6senses.com

THE WHOLE PREMISE

“Seeing is bot-hard.”



Hard for machines, easy for humans. This single assumption underlies the security of visual CAPTCHA.

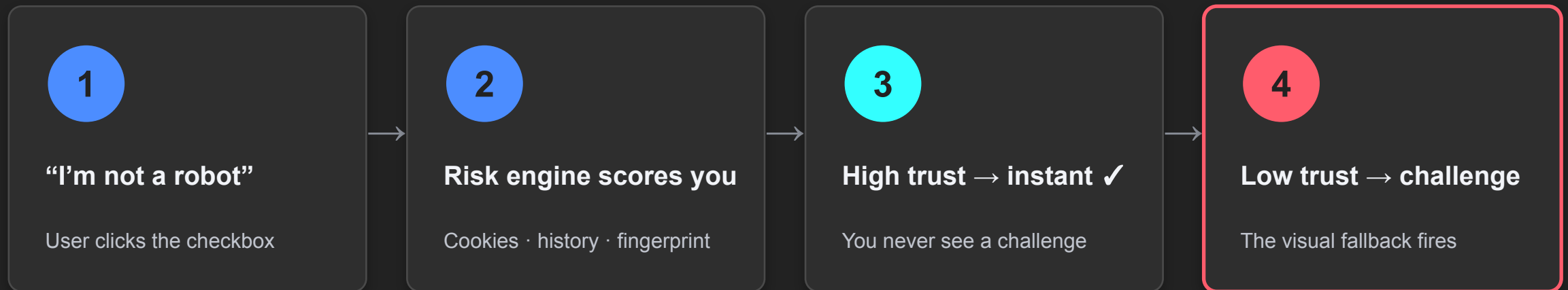
That assumption is dead.

Everyone showed it's solvable — with a catch.

Sivakorn 2016		Teoh 2025		This work	
EuroS&P / Black Hat Asia		USENIX Security		SummerCon 2026	
Model	CNN-based (cloud)	Model	GPT-4o (cloud)	Model	CLIP + OWLv2 (local)
Cost	Paid API (per call)	Cost	Paid (\$0.024 / challenge)	Cost	\$0 · no API
Code	API calls	Code	Agentic framework	Code	~200 LOC — or ZERO
Accuracy	70.78%	Accuracy	60.7%	Accuracy	92.6%*

** per-session success across 500 live sessions*

What once blocked bots is now the greatest vulnerability.



The irony: the part built to catch bots is the part a bot reliably beats. Hold that thought.

THE PLAN

Two phases: Measure offline, then attack online.

First: Are open source models good enough? **Then:** Does the attack actually get through?

PHASE 1 · OFFLINE

MEASURE

- WebDriver visits live reCAPTCHA sites
- Collects 1,000 challenge images + instructions
- Labeled for study + threshold tuning
- **Output:** models · accuracy · thresholds

→ **what works, and how well?**



Feed the winners

PHASE 2 · ONLINE (LIVE)

ATTACK

- WebDriver drives a real browser, live
- Solves reCAPTCHAs live using the selected models
- **Output:** real-world accuracy

→ **does it actually get through?**

Meet the two bosses.

TYPE A

3 × 3 · nine INDEPENDENT images



“Select all images with a fire hydrant”

Each tile is its own classification problem.

TYPE B

4 × 4 · ONE image, sliced up



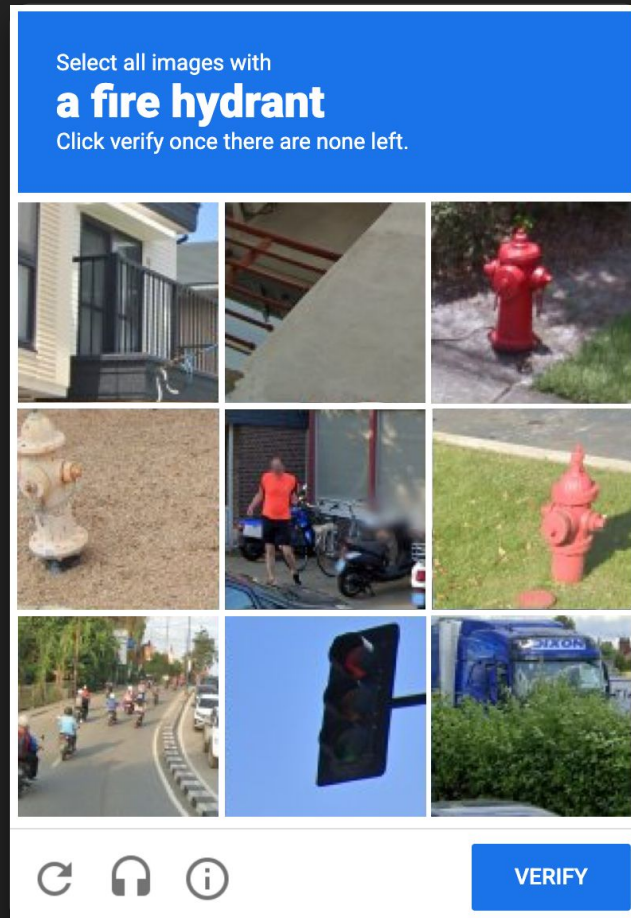
“Select all squares with traffic lights”

One object spread across tiles — spatial.

Nobody had named these. They need completely different solvers.

TYPE A

Nine independent images — two flavors.

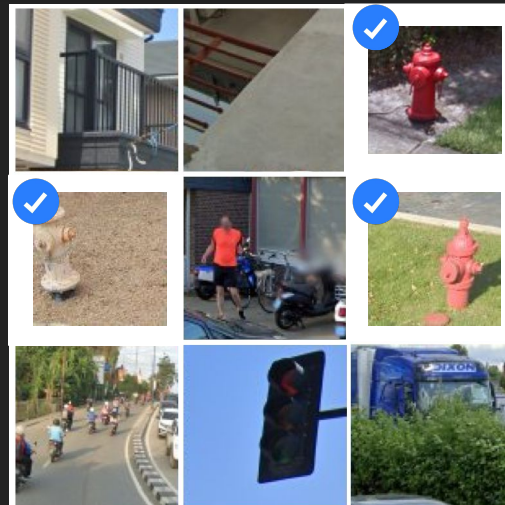


① STATIC

- No subtitle on the prompt
- Clicking a tile adds selection check mark
- Solve in a SINGLE pass → submit

② DYNAMIC

- “Click verify once there are none left”
- Clicking a tile refreshes it with a new image
- Loop: classify → click → wait → repeat

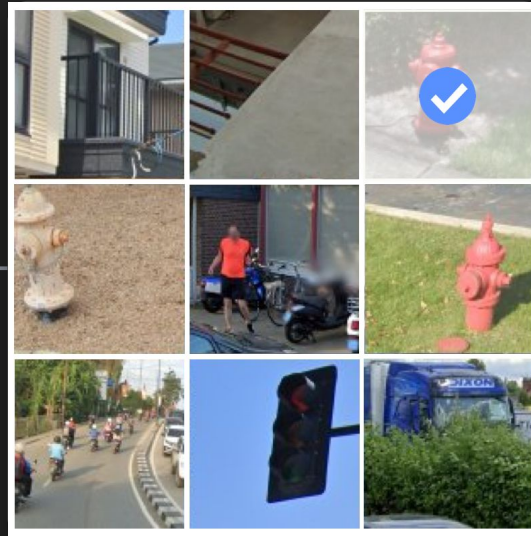


THE TRAP

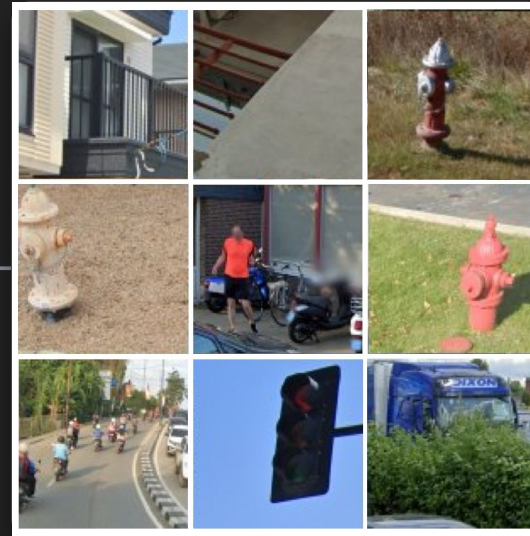
The refresh trap kills naive solvers.



1 · Classify the tile
in the grid



2 · Click the hydrant



3 · Tile refreshes —
new image

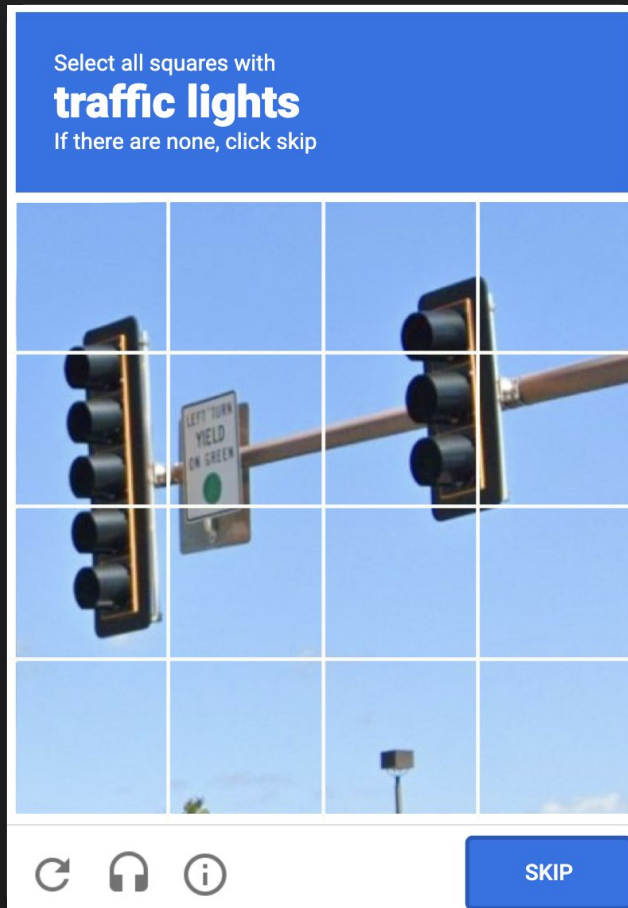
② DYNAMIC

Re-click a selected tile
and reCAPTCHA
deselects it.

A naive “classify twice”
pass undoes its own
work.

TYPE B

One image, sliced — now it's spatial.



Question:

“Which tiles contain the object?”

A tile can be 90% pole and 10% light.
Per-tile classification has no way to reason about that — you need detection.

Two models. Zero dollar\$. Your Local Machine.

Type A → CLIP ViT-H/14

- Zero-shot image–text matching
- “Does this tile match the object (e.g., a fire hydrant) ?”
- Perfect for nine independent tiles

Type B → OWLv2

- Open-vocabulary object detection
- Draws boxes around the objects you name
- Reasons about where an object sits

\$0 cost

Runs locally

No GPU needed

No training

No API key

CLIP: Does this image match this phrase?

Contrastive Language–Image Pre-training — 400M image–text pairs mapped into ONE shared space, so you can score any image against any words. Zero-shot, no training.



a photo of
"a fire hydrant"



shared embedding space



image & text vectors —
close together = a match



cosine similarity

0.34

one number: how close
the image is to the phrase
[-1.0, +1.0]

Why CLIP fits Type A

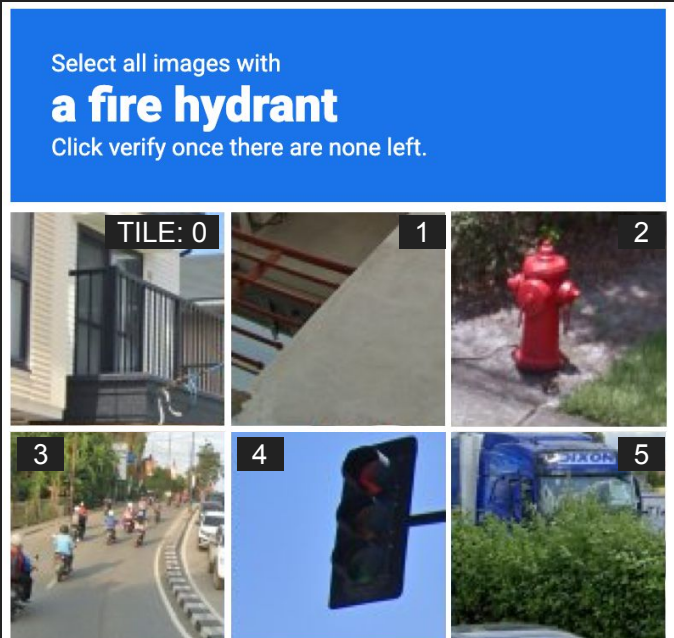
- Each of the 9 tiles is its own image
- Ask per tile: Does it match the keyword?
- Zero-shot — any keyword, no training
- No spatial reasoning (that's Type B / OWLv2)

One image + one phrase → one score. **To decide select vs. reject, we compare it against a fixed baseline phrase — next slide.**

The CLIP trick: Score it twice.

“Does it look like a hydrant” vs. “**more like a hydrant than like anything else.**”

`score = sim("a fire hydrant") - sim("something else")`



TILE 2 · Fire Hydrant			score +0.14 → SELECT
"a fire hydrant"		0.36	
"something else"		0.21	

TILE 3 · Motorcycle			score -0.07 → REJECT
"a fire hydrant"		0.10	
"something else"		0.17	

keyword "a fire hydrant" · select every tile with score ≥ 0.01

Bigger models barely move the needle.

Per-challenge exact-match accuracy on 662 Type A challenges — counts only if every correct tile is picked and no wrong ones.

ViT-B/32

86M params



52%

ViT-L/14

304M params



56%

ViT-H/14

632M params



58.5%

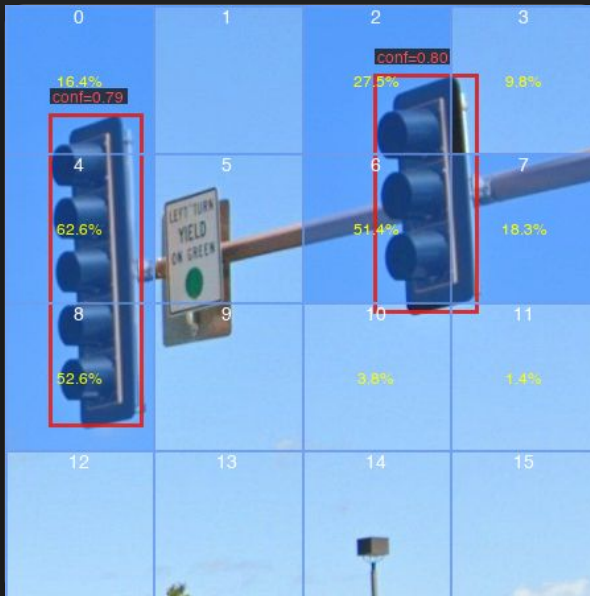
Why accuracy tops out at 58%

- Exact-match scoring — one wrong or missed tile fails the challenge
- So, a bigger model or threshold tuning gains a few points
- *Diminishing returns* — 7× the parameters (86M → 632M) buys just +6 points.

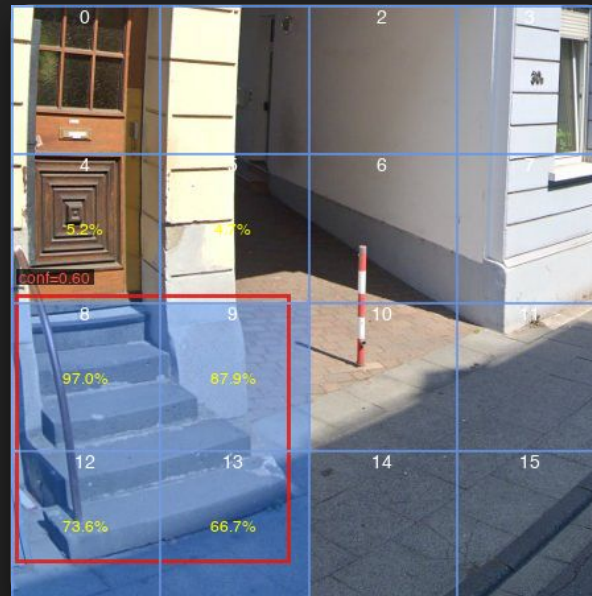
58% per challenge sounds mediocre — **but a bot just retry – and that changes everything***

OWLv2: Point an object detector at a word.

Open-vocabulary detection — give it any text query and it returns a bounding box + confidence for every match.
No training, no fixed list of classes.



"traffic lights" → 2 boxes
(conf 0.79, 0.80)



"stairs" → 1 box
(conf 0.60)

What OWLv2 hands you

- A query in plain words — any keyword, zero training
- A box + confidence for each object it finds
- The box = WHERE the object sits
- Several instances at once (two lights above)

Confidence gets it in. Coverage gets it clicked.

Every detection gets a confidence score. Every tile gets a coverage score.

① CONFIDENCE · detection

Is the object really there?

Every detection comes with a score from 0.0 to 1.0. We keep only the boxes above a confidence threshold (0.2) — so a shaky guess never triggers a click.

② COVERAGE · selection

Which tiles does it cover?

For each of the 16 tiles, we measure how much of the tile the box overlaps. Select every tile at or above the coverage threshold (15%).



keyword "traffic light" · $\text{conf} \geq 0.2$ and $\text{coverage} \geq 15\%$
Tile 2, 6 and 7 → selected

Confidence throws out false detections. **Coverage maps the surviving box onto the grid.**

OWLv2: Detect on the whole image, then map to tiles.

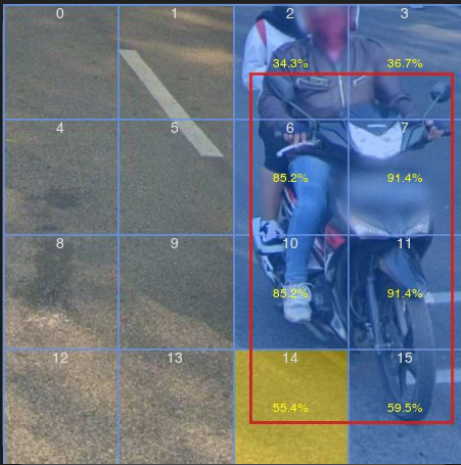
Per-challenge exact-match accuracy on 338 Type B challenges — counts only if every correct tile is picked and no wrong ones.



Original photo



① Detect box
(confidence)

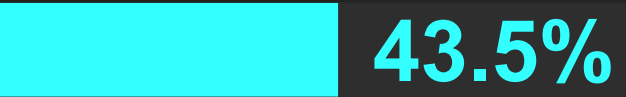


② tile coverage
(selection)

Per-tile CLIP vs. detection on Type B



CLIP, per-tile



OWLv2, detection

3.4× better

Detect on the full image → map each box's coverage onto the 4×4 grid → select tiles above the thresholds.

RETRY RETRY RETRY

“58%? That fails half the time.”

Humans get one shot. Bots get retries.

$$P(\geq 1 \text{ success in 5 attempts}) = 1 - (1 - p)^5$$

98.7%

at $p = 58\%$ (CLIP on Type A)

94.3%

at $p = 43\%$ (OWLv2 on Type B)

The real bar isn't accuracy.

Q: Can you afford to retry?

Bots always can.

How to make a bot look human?

500 live sessions · public demos & real websites (e.g., login, signup, comment)

Anti-detection

- Hide navigator.webdriver via CDP injection
- Strip Chrome automation tells (runtime, plugins)
- Make actions look human, not scripted
e.g., mouse: randomized offsets + timing delay

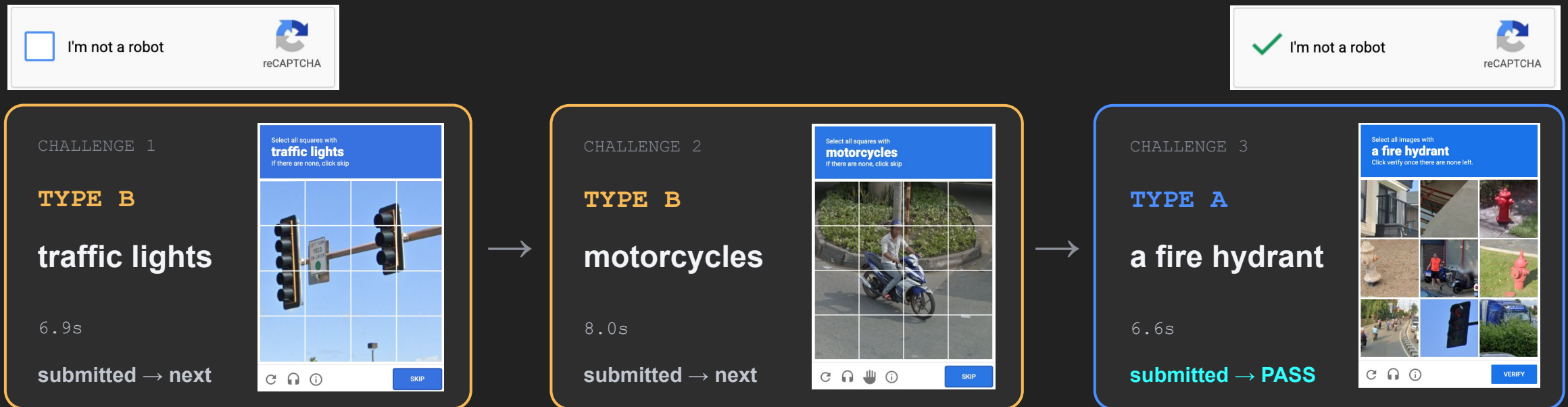
Honest controls (hardest case)

- No Google sign-in — zero account reputation
- No browsing history — no residual trust
- Fresh browser instance every session
- = exactly what a real bot faces

Reputation can hand a logged-in user a free pass — we refused that crutch, so the numbers are pure image-solving.

A session is many challenges, not one.

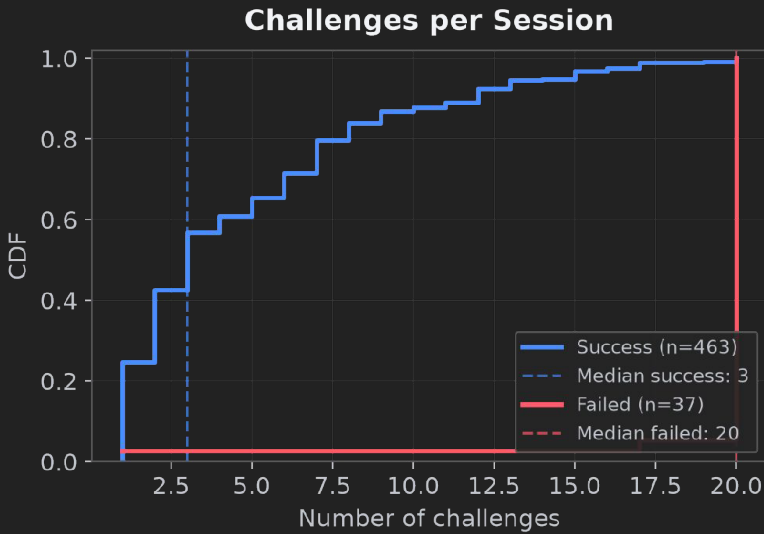
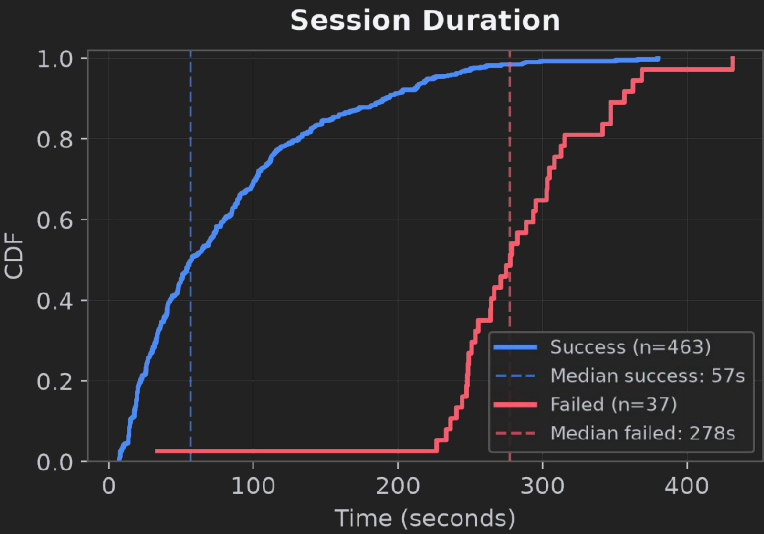
reCAPTCHA keeps serving challenges until its risk model is satisfied. Here's one real successful run.



3 challenges, 25.7s, one pass. **We score per session — and retries are why even a 58% solver wins.**

Fresh sessions — no login, no history — and it still gets in.

Online Solver Session Statistics (500 sessions)



92.6%

session success (463 / 500)

57s

median (success)

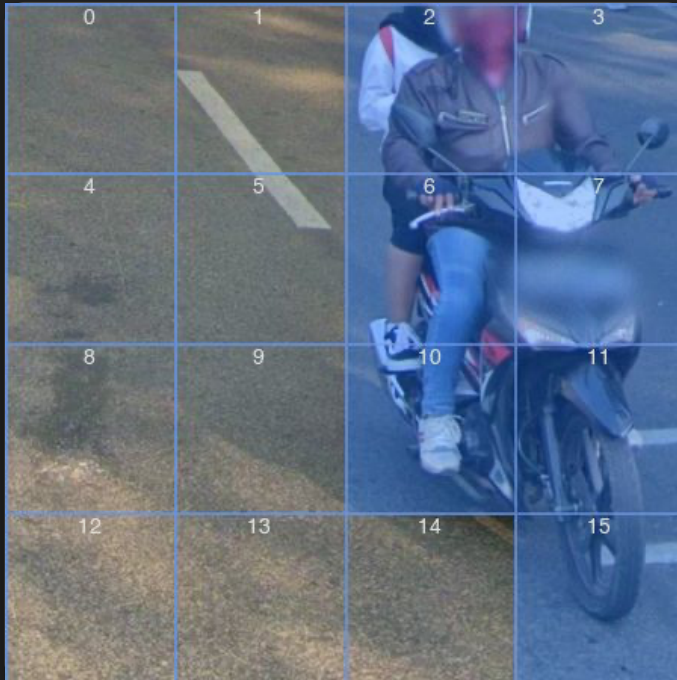
3

median challenges

Failures flatline at the wall:
20 challenges, 278s.

92.6% ONLINE VS. 58% OFFLINE?

In the real world, you can try multiple times.



“motorcycles” — does a correct tile include the rider? the shadow? reCAPTCHA accepts what a human would.

reCAPTCHA grades loosely

- Offline = strict exact-match: every right tile, zero wrong ones
- Online = accepts natural human interpretation of the keyword
- So, our offline accuracy is a conservative LOWER bound

...and you get many shots

- A session = several challenges + retries
- Per-challenge accuracy compounds into per-session success

Per-challenge, exact-match: 58.5% / 43.5%. Per-session, live: **92.6%**.

Now break it with zero code.

Swap the trained models for an AI assistant with eyes and a mouse — no CLIP, no OWLv2, no code.

```
/SKILL: challenge-solver.md
```

```
# Solve a reCAPTCHA challenge
1. Read the instruction; get the keyword.
2. "images" → Type A; "squares" → Type B.
3. Look at each tile. Does it show the keyword?
4. Click the matches. Submit.
```

Multimodal Agentic AI

Just plain English to a commodity AI assistant.

98%

offline accuracy (49 / 50 challenges)

96% Type A · 100% Type B

*Building a solver used to take real skill.
Now it just takes paragraphs.*

Zero-code is still limited by physics.

Prompt-only (zero code)

- 30–60s inference per challenge
- Dynamic Type A & long Type B sessions EXPIRE
- Killed by the ~2-min session timeout
- Trivially easy, but slow

Local models

- Tile classification in under 1 second
- Just grinds through retries
- Fast, scalable, \$0
- A little setup: just a Python script + model downloads

Pick your poison — either way, the defender loses.

Three things to take home.

01

Bots always win

Retries beat any per-challenge accuracy. 58% → 98.7% in five tries.

02

The skill floor is zero

You don't need to be a researcher. You need a chatbot and a few English sentences.

03

The fallback paradox

The challenge catches sessions that fail behavior analysis — i.e., the bot — then hands it a test it can pass.

The safety net is the trapdoor.



THANK YOU

I broke it in 2016. It's easier now.



Not a reCAPTCHA story — the story of every task we assumed only humans could do.

Suphanee Sivakorn

Data: <https://tinyurl.com/recaptcha-dataset> · Demo: <https://tinyurl.com/recaptcha-demo-vdo>

Questions? Let's talk crosswalks.